PROJECT REPORT

Submitted by

ALEENA JAMES

AJC23CS030

S6 CSE A BATCH

29

TITLE: AUTOMATIC TEXT SUMMARIZER

INDEX

1. Abstract 3
2. Problem Description 4
3. Libraries and tools used 5
4. System Design / Workflow 7
5. Implementation Details 8
6. Results and Output 9
7. Conclusion & Scope for Future Extension 11

ABSTRACT

In the modern digital era, vast amounts of textual information are produced and shared through online articles, research papers, reports, blogs, and various digital documents. As a result, individuals often face difficulty in quickly understanding and extracting important information from lengthy content. Reading large volumes of text can be time-consuming and inefficient, especially when users only need the key ideas or main points. Therefore, there is an increasing need for automated systems that can condense long texts into shorter, meaningful summaries while preserving the essential information.

This project proposes an AI-based text summarization system that automatically generates concise summaries from long text inputs. The application is developed using the Python programming language and provides a user-friendly graphical interface through **Streamlit**, allowing users to interact with the system easily through a web browser. Users can paste or enter large blocks of text into the application, and the system processes the input to produce a structured summary within seconds.

The summarization process is powered by **Google Gemini**, specifically the gemini-2.5-flash model, which analyzes the content and extracts the most relevant information. The generated summary highlights the main points of the original text, enabling users to quickly understand the core ideas without reading the entire document.

By combining artificial intelligence with an intuitive web interface, the proposed system provides an efficient solution for managing and understanding large amounts of textual information. The application helps reduce reading time, improves information accessibility, and demonstrates the practical use of AI technologies in real-world information processing tasks.

PROBLEM DESCRIPTION

In today's digital world, information is generated and shared at a very rapid pace through articles, research papers, news reports, blogs, and online documents. As a result, individuals are exposed to a vast amount of textual data on a daily basis. Reading and understanding these large volumes of information can be time-consuming and often overwhelming for users. Many people do not have the time to go through lengthy documents in detail, especially when they only need to understand the main ideas or key points.

Traditional methods of summarizing text involve manually reading the content and extracting the most important information. However, manual summarization requires considerable time, effort, and concentration. It can also lead to inconsistencies, as different individuals may summarize the same text in different ways depending on their interpretation. This makes manual summarization inefficient when dealing with large quantities of information.

With the advancement of Artificial Intelligence and Natural Language Processing (NLP), automated text summarization has become a practical solution to this problem. An automated system can analyze large amounts of text quickly and generate concise summaries that capture the essential ideas of the original content. Such systems help users quickly grasp the key information without reading the entire document.

Therefore, there is a need for an intelligent application that can automatically process lengthy text inputs and generate meaningful summaries in a short time. This project aims to address this need by developing a web-based text summarization system that allows users to enter long pieces of text and instantly obtain a clear and structured summary, thereby improving efficiency and reducing the time required to understand large textual content.

LIBRARIES AND TOOLS USED

1. Python

Purpose: Main programming language used to build the application.

Python is used to:

- Write the logic of the summarizer
- Connect the interface with the AI model
- Process user input and display output

2. Streamlit

Purpose: Used to create the **Graphical User Interface (GUI)** of the application.

Streamlit allows the Python script to run as a **web application** where users can interact with the system.

Used for:

- Creating the app layout
- Displaying titles and text
- Creating chat input boxes
- Showing summarized results
- Managing the sidebar settings

3. Google Gemini

Purpose: Performs the **actual text summarization using artificial intelligence**.

The Gemini model analyzes the input text and generates a **concise summary of the important points**.

In project

- Model used: **gemini-2.5-flash**
- It receives the text from the program
- Processes the content
- Generates a summarized output

4. Google GenAI Python SDK

Purpose: Acts as the **bridge between the Python program and the Gemini AI model**.

This library allows the program to send requests to Gemini and receive responses.

Functions

- Connects to Gemini API
- Sends input text
- Receives AI-generated summary

5. Custom CSS

Purpose: Improves the **visual design of the interface**.

It is used to:

- Change background colors
- Style chat messages
- Modify fonts and layout
- Hide Streamlit branding

SYSTEM DESIGN /WORKFLOW

The system follows a simple workflow where user input is processed by the application and summarized using an AI model. The interface is developed using **Streamlit**, while the summarization is generated using **Google Gemini**.

Step 1: User Input

The user opens the web application and enters or pastes a block of text into the chat input box provided in the interface. The interface is designed using Streamlit, which allows users to interact with the application easily through a browser.

Step 2: Input Processing

Once the text is submitted, the application receives the input and prepares it for processing. The system formats the text into a prompt that instructs the AI model to generate a concise and structured summary.

Step 3: AI Model Request

The formatted prompt is sent to the Gemini AI model through the API connection established in the Python program. The AI model analyzes the content and identifies the most important information from the text.

Step 4: Summary Generation

The Gemini model processes the input and generates a summarized version of the text. The summary may include key points or bullet points depending on the prompt instructions.

Step 5: Display Output

The generated summary is returned to the application and displayed in the chat interface. The user can immediately view the summarized content.

Step 6: Conversation Management

The system stores the interaction in the session state so that previous messages remain visible in the conversation history. Users can also clear the chat using the “Clear Conversation” option in the sidebar.

IMPLEMENTATION DETAILS

The Text Summarizer application is implemented using Python and a web-based interface. The system integrates a graphical interface with an AI model to generate concise summaries from long text inputs. The application follows a modular structure where each component performs a specific function.

1. Application Interface

The user interface is developed using **Streamlit**, which allows the Python script to run as an interactive web application. Streamlit components such as st.title(), st.chat_input(), st.sidebar, and st.chat_message() are used to create the layout, input fields, and chat-style interaction. Custom CSS is also included to enhance the visual appearance by modifying colors, fonts, and message containers.

2. AI Model Integration

The summarization functionality is powered by **Google Gemini**. The application connects to the model using the **Google GenAI Python SDK**, which allows the program to send text input and receive AI-generated summaries.

The connection is established using an API key, and the model used in this project is **gemini-2.5-flash**, which is optimized for fast responses.

3. Prompt Construction

When a user submits text, the system automatically formats the input into a prompt instructing the AI model to generate a concise and structured summary. This ensures the output highlights the key points of the provided text.

4. Response Handling

After processing the prompt, the Gemini model returns the generated summary. The application captures this response and displays it in the chat interface. If an error occurs or the response is empty, the system displays an error message to inform the user.

5. Session Management

The application maintains conversation history using Streamlit's session_state. This allows previous messages and summaries to remain visible during the session, creating a chat-like experience. Users can also clear the conversation using a button in the sidebar.

6. Output Display

The final summary generated by the AI model is displayed in the assistant chat window. The output is formatted using Markdown to present the summary clearly and make it easier for users to read.

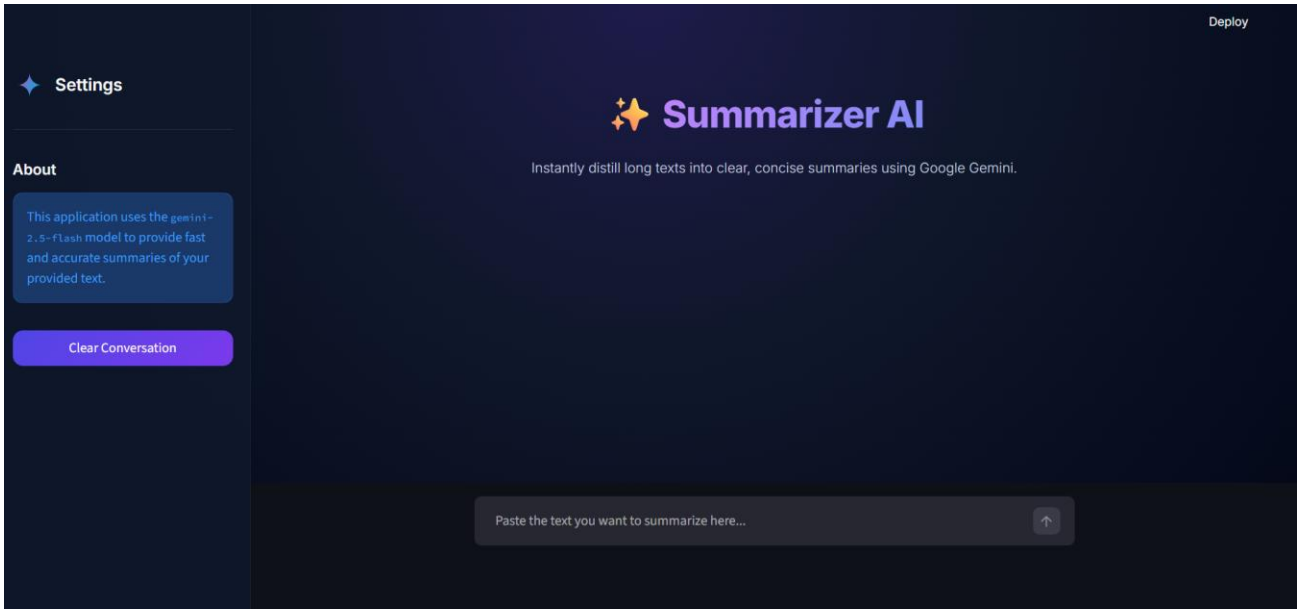
RESULT AND OUTPUT

The developed Text Summarizer application successfully generates concise summaries from long textual inputs through an interactive web interface. The system was tested with different types of text such as articles, paragraphs, and informational content. In each case, the application was able to analyze the input and produce a clear summary containing the key points of the original text.

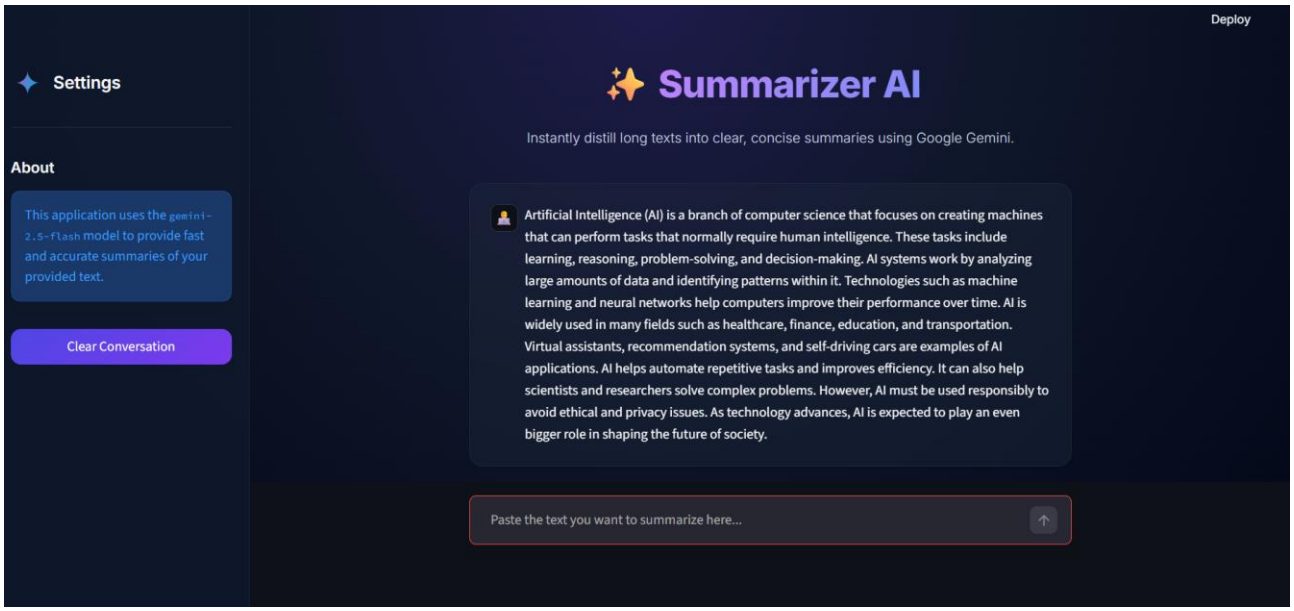
The graphical interface built using **Streamlit** allows users to easily enter text and view the generated summaries in a chat-style format. After the user submits the input text, the system processes it and sends a request to the **Google Gemini** model, which generates a concise summary. The result is then displayed instantly on the screen.

The output produced by the system is structured and readable, often highlighting the main ideas of the text in a shortened format. This demonstrates that the system effectively reduces lengthy content while preserving the essential information, thereby helping users quickly understand large amounts of text.

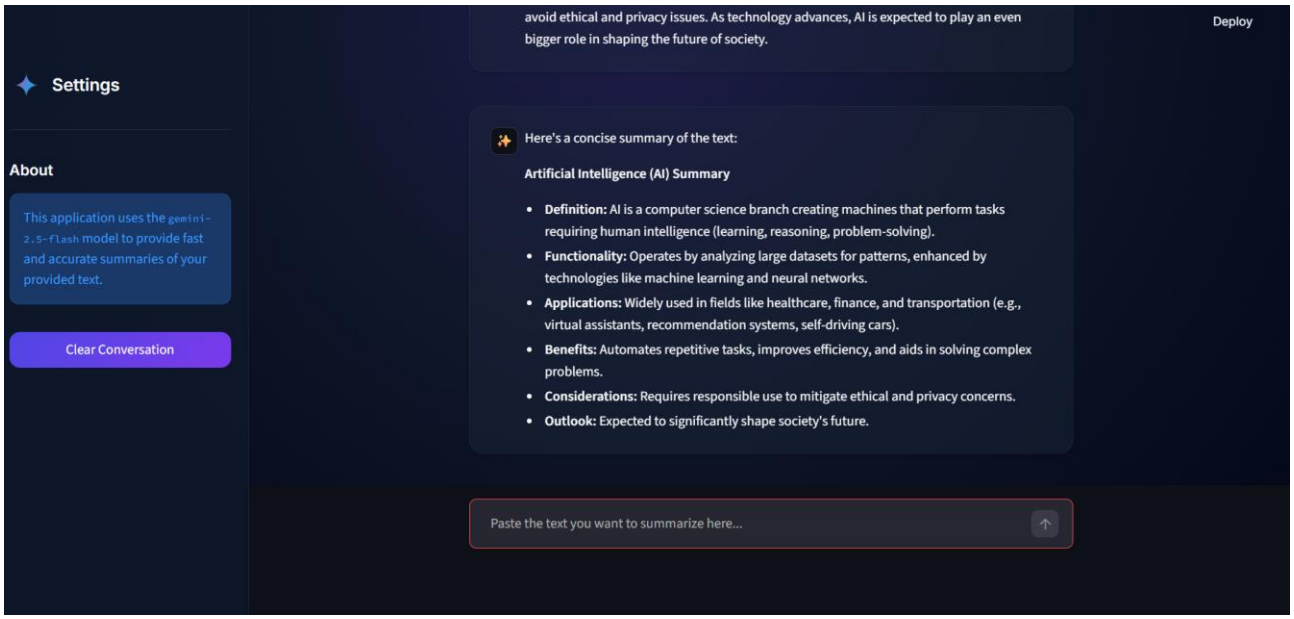
INTERFACE:



INPUT:



OUTPUT:



CONCLUSION

The Text Summarizer application was successfully developed to automatically generate concise summaries from long text inputs. The system provides an easy-to-use web interface that allows users to quickly obtain key information from large blocks of text. By integrating **Streamlit** for the user interface and **Google Gemini** for text processing, the application demonstrates how artificial intelligence can simplify information consumption. The system effectively reduces reading time while preserving the important points of the original content. Overall, the project shows how AI-powered tools can assist users in quickly understanding large amounts of textual data.

SCOPE FOR FUTURE EXTENSION

The system can be further enhanced by adding several additional features to improve functionality and usability.

1. **File Upload Support**
   The application can be extended to allow users to upload documents such as PDF, Word, or text files for summarization.

2. **Multi-Language Summarization**
   Future versions can support summarizing content in multiple languages, making the system accessible to a wider range of users.

3. **Custom Summary Length**
   Users could be given options to select short, medium, or detailed summaries based on their needs.

4. **Voice Input and Output**
   Speech recognition and text-to-speech features could be integrated to allow users to speak the input text and listen to the summarized output.

5. **Integration with Web Articles**
   The system could be enhanced to summarize content directly from URLs or online articles.

6. **Improved AI Models**
   More advanced AI models or domain-specific summarization models can be integrated to produce even more accurate summaries.